

PA1 – Secure Real-Time Media Streaming Project

Alexandre Santos 72970

Miguel Mestre 73018

Contents

1	Introduction	3
2	System Architecture	3
2.1	Main Components	3
2.2	Network Flow	3
3	Part 1 – RTSSP Design and Implementation	3
3.1	Objective	3
3.2	Part 1 Security Goals	4
3.3	RTSSP Packet Format	4
3.4	RTSSP Message Types	4
3.5	Cryptographic Algorithms	4
3.6	Movie Request Flow	5
3.7	Replay Protection	5
3.8	Part 1 Runtime Flow	5
4	Part 2 – SHP Design and Implementation	6
4.1	Objective	6
4.2	SHP Protocol Flow	6
4.2.1	SHP_CLIENT_HELLO	6
4.2.2	SHP_SERVER_HELLO	7
4.2.3	SHP_CLIENT_FINAL	7
4.3	ECDH Key Establishment	7
4.4	Session Key Derivation	8
4.5	Ciphersuite Negotiation	8
4.6	ECDSA Signatures	8

5	Expected Runtime Output	9
5.1	Server Output	9
5.2	Proxy Output	9
6	Testing and Validation	10
6.1	Functional Tests	10
6.1.1	Part 1 – RTSSP Tests	10
6.1.2	Part 2 – SHP + RTSSP Tests	10
6.2	Security Tests	11
6.2.1	Packet Tampering	11
6.2.2	Replay Attack	12
6.2.3	Certificate Substitution	12
6.2.4	Invalid Signature	12
7	Statistics and Experimental Results	12
7.1	Part 1 – RTSSP with Static Keys	12
7.1.1	Server Statistics	12
7.1.2	Proxy Statistics	12
7.2	Part 2 – SHP + RTSSP with Dynamic Keys	13
7.2.1	Server Statistics	13
7.2.2	Proxy Statistics	13
8	Conclusion	13

1 Introduction

The goal of this project is to secure a real-time media streaming system originally based on UDP datagrams carrying MPEG4 encoded media frames. The initial system consisted of a streaming server, a proxy or box, and a media player such as VLC or MPV. In the initial version, UDP packets carried media frames in cleartext, which made the system vulnerable to packet sniffing, data leakage, message tampering, and replay attacks.

The implemented solution protects the UDP stream at the application level. In Part 1, the solution introduces RTSSP, a secure record-like protocol inspired by TLS/DTLS record protection. In Part 2, the solution introduces SHP, a handshake protocol inspired by TLS/DTLS handshakes, responsible for dynamic ciphersuite negotiation, mutual authentication, and key establishment.

2 System Architecture

2.1 Main Components

The system is composed of three main components:

- **Streaming Server:** stores the media files and sends the media frames in real time.
- **Secure UDP Proxy:** receives protected packets, verifies and decrypts them, and forwards plaintext frames locally.
- **Media Player:** receives the local UDP stream and plays the video. VLC was used for testing.

2.2 Network Flow

The general flow is:



In Part 1, the server and proxy use static symmetric keys from `Cryptoconfig.conf`. In Part 2, SHP derives fresh session keys before streaming begins.

3 Part 1 – RTSSP Design and Implementation

3.1 Objective

The objective of Part 1 is to transform the original unprotected UDP stream into a protected real-time stream. This is achieved by encrypting each media frame before transmission and verifying it at the proxy before forwarding it to the media player.

3.2 Part 1 Security Goals

RTSSP provides the following properties:

- **Confidentiality:** media frames are encrypted before being transmitted.
- **Integrity:** modified packets are detected and rejected.
- **Data-origin authentication:** packets can only be authenticated by an entity that knows the symmetric key.
- **Replay protection:** sequence numbers are used to detect repeated or old packets.
- **Session control:** explicit START, DATA, END, and ERROR message types were added.

3.3 RTSSP Packet Format

Each RTSSP packet has a header and a protected payload. The implemented format is:

Field	Size	Description
Type	1 byte	START, DATA, END, ERROR
Version	2 bytes	RTSSP protocol version
Sequence Number	4 bytes	Monotonic packet number
Ciphertext Length	4 bytes	Length of encrypted payload
IV Length	1 byte	Length of nonce/IV
MAC Length	1 byte	Length of MAC, if used
IV / Nonce	variable	Random value used by cipher
Ciphertext	variable	Encrypted payload
MAC	variable	HMAC for non-AEAD ciphersuites

3.4 RTSSP Message Types

The following message types are supported:

Message Type	Purpose
TYPE_START	Marks the beginning of a streaming session
TYPE_DATA	Carries encrypted media frame data
TYPE_END	Marks the end of a streaming session
TYPE_ERROR	Carries error information

3.5 Cryptographic Algorithms

The implementation supports the following ciphersuites:

- AES/GCM/NoPadding
- ChaCha20-Poly1305

- AES/CTR/NoPadding with HmacSHA256
- AES/CBC/PKCS5Padding with HmacSHA256

AES-GCM and ChaCha20-Poly1305 are AEAD modes, meaning that they provide encryption and integrity in a single operation. AES-CTR and AES-CBC are not AEAD modes, so the implementation protects them with an external HMAC. In Part 1, all four configurations were tested successfully using static keys from `Cryptoconfig.conf`.

3.6 Movie Request Flow

The proxy initiates the stream by sending a request to the server:

```
REQUEST: cars.dat:localhost:8888
```

The fields are:

- Requested movie name: `cars.dat`
- Proxy RTSSP listening endpoint: `localhost:8888`

If the movie exists and a cryptographic configuration is available, the server starts streaming.

3.7 Replay Protection

Each RTSSP packet carries a sequence number. The proxy stores the last accepted sequence number and rejects packets with a sequence number lower than or equal to the last accepted one.

This prevents simple replay attacks where an attacker resends an old valid encrypted packet.

3.8 Part 1 Runtime Flow

The Part 1 flow is:

1. Proxy reads `config.properties`.
2. Proxy sends a movie request to the server.
3. Server validates the movie and loads `Cryptoconfig.conf`.
4. Server sends RTSSP START.
5. Server sends encrypted RTSSP DATA packets.
6. Proxy verifies and decrypts each packet.
7. Proxy forwards plaintext frames to VLC on port 7777.
8. Server sends RTSSP END.
9. Server and proxy print statistics.

4 Part 2 – SHP Design and Implementation

4.1 Objective

The objective of Part 2 is to remove the need for pre-shared static keys. SHP dynamically negotiates the ciphersuite and establishes session keys before RTSSP streaming begins.

Part 2 provides:

- Dynamic ciphersuite negotiation
- ECDH-based key establishment
- ECDSA signatures
- Certificate-based mutual authentication
- Encrypted final handshake message
- RTSSP streaming using derived session keys

4.2 SHP Protocol Flow

The implemented SHP flow is:

```
Proxy  → Server: SHP_CLIENT_HELLO
Server → Proxy: SHP_SERVER_HELLO
Proxy  → Server: encrypted SHP_CLIENT_FINAL
Server → Proxy: RTSSP protected stream
```

4.2.1 SHP_CLIENT_HELLO

The proxy sends the first SHP message. It contains:

- Requested movie name
- Proxy RTSSP stream endpoint
- Supported ciphersuites
- Proxy ECDH public key
- Client nonce
- Proxy certificate
- ECDSA signature

Conceptual format:

```
SHP_CLIENT_HELLO |
movieName |
streamEndpoint |
supportedCipherSuites |
clientECDHPublicKey |
clientNonce |
```

```
clientCertificate |
signature
```

The signature is computed over all fields except the signature itself.

4.2.2 SHP_SERVER_HELLO

The server replies with:

- Confirmed movie name
- Selected ciphersuite
- Server ECDH public key
- Server nonce
- Response to client nonce
- Server certificate
- ECDSA signature

Conceptual format:

```
SHP_SERVER_HELLO |
movieName |
selectedCipherSuite |
serverECDHPublicKey |
serverNonce |
clientNonceResponse |
serverCertificate |
signature
```

4.2.3 SHP_CLIENT_FINAL

After verifying the server response, the proxy derives the same session keys and sends an encrypted final message:

```
SHP_CLIENT_FINAL | serverNonceResponse | START_STREAM
```

This message is protected using RTSSP and the newly derived session key. If the server successfully decrypts this message and verifies the server nonce response, the RTSSP stream begins.

4.3 ECDH Key Establishment

Both the proxy and the server generate an ephemeral EC key pair using the `secp256r1` curve.

The shared secret is computed as:

```
Proxy: ECDH(proxyPrivateKey, serverPublicKey)
Server: ECDH(serverPrivateKey, proxyPublicKey)
```

Both computations produce the same shared secret.

4.4 Session Key Derivation

The shared secret is not used directly as an encryption key. Instead, a key derivation step is applied.

The implemented derivation uses:

```
SHA-256(sharedSecret || clientNonce || serverNonce || label)
```

Different labels are used for different keys:

- RTSSP ENCRYPTION KEY
- RTSSP MAC KEY

For AES and ChaCha20, the first 32 bytes are used as a 256-bit symmetric key.

4.5 Ciphersuite Negotiation

The proxy sends a list of supported ciphersuites in `SHP_CLIENT_HELLO`. The server compares that list with its own list and selects the first mutually supported option.

Example proxy configuration:

```
supportedCiphersuites=AES/GCM/NoPadding,ChaCha20-Poly1305,AES/CTR/NoPadding
```

The server currently supports:

```
AES/GCM/NoPadding
ChaCha20-Poly1305
AES/CTR/NoPadding
```

For AES/GCM/NoPadding and ChaCha20-Poly1305, no separate MAC key is needed because they are AEAD modes. For AES/CTR/NoPadding, SHP derives both an encryption key and a MAC key, and RTSSP uses HmacSHA256 for integrity protection.

4.6 ECDSA Signatures

The proxy signs the `SHP_CLIENT_HELLO` using its private key. The server verifies the signature using the public key from the proxy certificate.

The server signs the `SHP_SERVER_HELLO` using its private key. The proxy verifies the signature using the public key from the server certificate.

The signature algorithm is:

```
SHA256withECDSA
```

This gives the handshake integrity and authenticates the identity of each endpoint.

5 Expected Runtime Output

5.1 Server Output

Expected server output for Part 2:

```
Server waiting for signed SHP CLIENT_HELLO on port 9999
SHP CLIENT_HELLO received
Proxy certificate trusted
CLIENT_HELLO signature verified
SHP SERVER_HELLO sent and signed
Selected CipherSuite: AES/GCM/NoPadding
Session keys derived
SHP CLIENT_FINAL received and verified
SHP handshake completed with mutual authentication
Starting RTSSP stream...
RTSSP START sent
::::::::::::::::::::::::::::::::::
RTSSP END sent
DONE! Frames sent: ...
```

5.2 Proxy Output

Expected proxy output:

```
Proxy listening for RTSSP stream on localhost:8888
Starting SHP handshake with mutual authentication...
SHP CLIENT_HELLO sent and signed
SHP SERVER_HELLO received
Server certificate trusted
SERVER_HELLO signature verified
Selected CipherSuite: AES/GCM/NoPadding
Session keys derived
SHP CLIENT_FINAL sent encrypted
SHP handshake completed
Waiting for RTSSP stream...

RTSSP START received: START:cars.dat
```

```

.....
RTSSP END received: END:...
Frames received: ...
Packets dropped: 0

```

6 Testing and Validation

6.1 Functional Tests

The implementation was tested in both Part 1 and Part 2 using `cars.dat` and `monsters.dat`. In Part 1, RTSSP was tested with static cryptographic configuration from `Cryptoconfig.conf`. In Part 2, SHP was tested first to establish the session keys dynamically, and then RTSSP was executed using those derived keys.

6.1.1 Part 1 – RTSSP Tests

In Part 1, the four supported RTSSP ciphersuite configurations were tested. All of them worked correctly.

Test	Expected Result	Status
Use AES/GCM/NoPadding	Stream decrypts correctly	Passed
Use ChaCha20-Poly1305	Stream decrypts correctly	Passed
Use AES/CTR/NoPadding with HmacSHA256	Stream decrypts correctly and HMAC verifies	Passed
Use AES/CBC/PKCS5Padding with HmacSHA256	Stream decrypts correctly and HMAC verifies	Passed
Replay old packet	Proxy rejects repeated packet through sequence number check	Implemented
Tamper packet	Proxy rejects invalid packet dur- ing authentication/decryption	Implemented

6.1.2 Part 2 – SHP + RTSSP Tests

In Part 2, the SHP handshake was implemented and tested. The proxy and server successfully performed mutual authentication, negotiated a ciphersuite, established an ECDH shared secret, derived RTSSP session keys, and then used those keys to protect the RTSSP stream.

The ciphersuites tested in Part 2 were **AES/GCM/NoPadding**, **ChaCha20-Poly1305**, and **AES/CTR/NoPadding** with **HmacSHA256**. All tested configurations worked correctly.

Test	Expected Result	Status
SHP ciphersuite negotiation	Server selects a mutually supported ciphersuite	Passed
ECDH key establishment	Server and proxy derive the same shared secret	Passed
ECDSA signatures	SHP messages are verified correctly	Passed
Certificate-based mutual authentication	Server and proxy authenticate each other	Passed
Encrypted SHP_CLIENT_FINAL	Server decrypts and verifies final message	Passed
Use AES/GCM/NoPadding	Stream decrypts correctly using derived keys	Passed
Use ChaCha20-Poly1305	Stream decrypts correctly using derived keys	Passed
Use AES/CTR/NoPadding with HmacSHA256	Stream decrypts correctly and HMAC verifies using derived keys	Passed
Replay old packet	Proxy rejects repeated packet through sequence number check	Implemented
Tamper packet	Proxy rejects invalid packet during authentication/decryption	Implemented
Wrong certificate	Handshake fails	Implemented
Wrong signature	Handshake fails	Implemented

6.2 Security Tests

6.2.1 Packet Tampering

If an attacker modifies the ciphertext of an RTSSP packet, the proxy rejects the packet. For AEAD ciphersuites such as **AES/GCM/NoPadding** and **ChaCha20-Poly1305**, authentication fails during decryption. For **AES/CTR/NoPadding**, the packet is rejected because the HMAC verification fails.

6.2.2 Replay Attack

If an old RTSSP packet is replayed, its sequence number is lower than or equal to the last accepted sequence number. The proxy detects this and rejects the packet.

6.2.3 Certificate Substitution

In Part 2, if a peer sends a certificate that is not present in the corresponding truststore, the handshake fails. This prevents an untrusted proxy or server from completing the SHP handshake.

6.2.4 Invalid Signature

If a signed SHP message is modified, the ECDSA signature verification fails and the handshake is rejected.

7 Statistics and Experimental Results

Both server and proxy print statistics at the end of each streaming session. The server measures the number of frames sent and the sending throughput, while the proxy measures the number of frames received, dropped packets, and receiving throughput.

7.1 Part 1 – RTSSP with Static Keys

This test was performed using `cars.dat` with the static cryptographic configuration defined in `Cryptoconfig.conf`.

7.1.1 Server Statistics

Metric	Observed Value
Frames sent	30515
Movie duration	128 s
Throughput	238 fps
Throughput	2509 Kbps

7.1.2 Proxy Statistics

Metric	Observed Value
Frames received	30515
Packets dropped	0
Throughput	238 fps

The number of frames sent by the server matches the number of frames received by the proxy. The proxy reported zero dropped packets, meaning that all RTSSP packets were successfully received, authenticated, decrypted, and forwarded to the media player.

7.2 Part 2 – SHP + RTSSP with Dynamic Keys

This test was performed using `cars.dat`. In this version, the server and proxy first executed the SHP handshake. After the handshake, RTSSP used the dynamically derived session keys to protect the stream.

7.2.1 Server Statistics

Metric	Observed Value
Frames sent	30515
Movie duration	128 s
Throughput	238 fps
Throughput	2509 Kbps

7.2.2 Proxy Statistics

Metric	Observed Value
Frames received	30515
Packets dropped	0
Throughput	238 fps

The Part 2 results are consistent with the Part 1 results. The same number of frames was sent and received, with zero dropped packets. This shows that the SHP handshake correctly established the session keys and that RTSSP was able to use those keys without affecting the streaming behavior in this test.

8 Conclusion

This project successfully transforms a basic UDP real-time media streaming system into a secure streaming system. In Part 1, RTSSP protects media frames using symmetric encryption, integrity verification, explicit protocol message types, and replay protection. In Part 2, SHP extends the system with dynamic session key establishment, ciphersuite negotiation, ECDH, ECDSA signatures, certificates, keystores, truststores, and mutual authentication.

The resulting stack, SHP/RTSSP/UDP, provides a simplified but functional secure streaming protocol inspired by TLS/DTLS concepts, adapted to the specific requirements of real-time UDP media streaming.